

GraphWiz 中文精读导读：教大模型 “一步一步” 解图论题的开源专家

原文：Nuo Chen, Yuhan Li, Jianheng Tang, Jia Li (KDD 2024)

本导读为面向初学者的中文学习笔记：含要点翻译、公式精讲与图表解读。

论文卡片

标题	GraphWiz: An Instruction-Following Language Model for Graph Computational Problems
作者	Nuo Chen, Yuhan Li, Jianheng Tang, Jia Li (陈诺、李宇晗、唐建衡、李佳)
机构	香港科技大学 (广州) 数据科学与分析学域 / 香港科技大学
会议	KDD 2024 (ACM SIGKDD 知识发现与数据挖掘会议)
arXiv	2402.16029
项目页	https://graph-wiz.github.io
代码	https://github.com/nuochenpku/Graph-Reasoning-LLM
方向	图 (Graph) × 大模型 × 指令微调 / 偏好对齐 (DPO)
一句话	先造一个带“显式推理过程”的大规模图论指令数据集 GraphInstruct, 再用它微调出能解 9 类图论题、且超过 GPT-4 的开源模型 GraphWiz。

1 一、这篇论文在解决什么问题

大语言模型 (LLM, 如 GPT-4、LLaMA) 在文字处理上已经很强, 但作者发现: 让它们真正**读懂一张图、并算出图论问题的答案**, 这件事还很少有人系统研究。这里说的“图”是数学意义上的图——由**节点 (node)** 和**边 (edge)** 组成的结构, 比如社交网络、电路、地图路网。而“图计算问题 (graph computational problem)”指的是要靠图算法才能解决的题目, 例如: 两个节点之间有没有路径? 最短路是多少? 图里有没有环?

已有做法的三个痛点 (作者在引言里点名):

1. **只会“猜”, 不会“讲”**: 以往多用精心设计的提示词 (prompt) 去“哄”GPT-4 这类闭源模型答题, 但模型只给一个 yes/no, 看不到推理过程——你无法判断它是真的推理出来, 还是蒙对了。

2. **开源模型很弱**：同样的提示法，开源的 LLaMA 表现远不如 GPT-4；而且把每条边写成一句话会让输入变得很长，限制了能处理的图规模（以往研究普遍把图限制在 20 个节点、100 条边以内）。
3. **缺一套“训练教材”**：市面上没有一个开源数据集，能教模型把图论题的推理过程一步步写出来。

本文目标：训练**单独一个**开源模型，让它既能解多种类型、多种规模的图论问题，又能像图 1 那样用**自然语言把推理路径明明白白写出来**。为此作者做了三件事：造数据集 GraphInstruct；用它微调出 GraphWiz；再用 DPO 进一步对齐，得到更强的 GraphWiz-DPO。

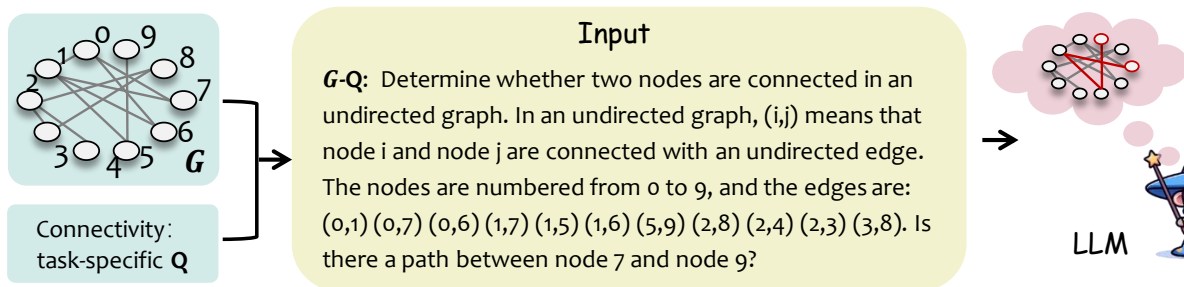


图 1：原文 Figure 1：用 LLM “显式地”在自然语言里求解连通性（*Connectivity*）任务的例子（图内英文为原文保留）。输入是一张无向图和一个问题“节点 7 和节点 9 之间是否有路径？”，模型不仅回答“yes”，还写出推理链：节点 7 连到节点 1、节点 1 连到节点 5、节点 5 连到节点 9，于是路径为 $[7 \rightarrow 1 \rightarrow 5 \rightarrow 9]$ 。这种“把思路写出来”正是本文追求的能力。

2 二、摘要中译

大语言模型 (LLM) 已在众多领域取得令人瞩目的成功，但它们**理解与求解复杂图问题**的能力却鲜有探索。为弥补这一空白，我们提出 **GraphInstruct**——一个新颖的指令微调数据集，旨在让语言模型能够通过**显式的推理路径**应对各类图问题。基于 GraphInstruct，我们构建了 **GraphWiz**：一个开源语言模型，能够在求解多种图计算问题的同时生成清晰的推理过程。为进一步提升模型的性能与可靠性，我们在图问题求解的情境中引入了**直接偏好优化 (Direct Preference Optimization, DPO)** 框架。改进后的模型 **GraphWiz-DPO** 在九个不同复杂度的任务上取得了 **65%** 的平均准确率，超过了平均准确率为 **43.8%** 的 GPT-4。我们的研究还考察了训练数据量与模型性能之间的关系，强调了随数据量增大而产生的**过拟合风险**；并探索了模型在不同任务、不同数据集间的**可迁移性**，展示了其稳健的零样本泛化能力。GraphWiz 为开发专精于图推理与图问题求解的 LLM 提供了新的蓝图与有价值的洞见。

3 三、三大核心贡献

1. **首个大规模图论指令数据集 GraphInstruct**：覆盖 9 类图计算任务，且每条样本都带**显式推理路径**，让训练出的模型既能给答案、又能给过程，大幅提升可解释性。

2. **首个图论专精开源模型 GraphWiz**：能解多类型、多规模图问题，性能显著超过当前最强闭源模型 GPT-4。
3. **深入的影响因素分析**：细致分析了训练数据量、DPO 中“负样本”采样策略等对性能的影响，为后续模型优化提供洞见（尤其揭示了“数据越多不一定越好、存在过拟合阈值”这一现象）。

4 四、预备知识（小白先看这里）

【小白补丁】什么是指令微调（Instruction Tuning）与 SFT

指令微调 = 用大量“指令（自然语言任务描述）+ 标准答案”的样本去训练 LLM，让它学会**听懂人话、按要求做事**。本文用的具体训练方式叫 **SFT（Supervised Fine-Tuning，监督微调）**：把“图 + 问题”当输入、把“正确的推理路径”当标准输出，用最普通的“预测下一个词”的语言模型损失来训练。一句话：**照着标准答案抄**，学会模仿专家的解题写法。

【小白补丁】什么是 DPO（直接偏好优化）

SFT 只教模型“什么是对的”，但不教它“主动避开什么是错的”。**DPO（Direct Preference Optimization）**就是用来补这一课的。它给模型成对的样本：一个**更受偏好的回答** R_w （win，正确推理链）和一个**不受偏好的回答** R_l （lose，错误推理链），训练目标是让模型**相对地更喜欢** R_w 、**更不喜欢** R_l 。它是 RLHF（基于人类反馈的强化学习）的一个轻量替代：RLHF 要训练 4 个子模型、又贵又复杂，而 DPO 把它简化成一个可以直接优化的损失函数，不需要单独训练奖励模型。

【小白补丁】什么是图论里的“复杂度”与这 9 类任务的难度

计算机里用**时间复杂度**衡量一个算法要算多久。本文把 9 类任务按难度分三档：**线性复杂度** ($O(|V| + |E|)$)，最快，如判连通、找环）、**多项式复杂度**（稍慢，如最短路、最大流）、以及 **NP 完全（NP-Complete）**——目前没有已知的高效算法、规模一大就极难算（如哈密顿路径、子图匹配）。 $|V|$ 、 $|E|$ 分别表示图里节点数、边数。难度越高，越能考验模型是不是“真会推理”。

形式化记号（看不懂可跳过，用到时再回看）：一张无向或有向图记作 $G = \{V, E\}$ ， V, E 分别是节点集与边集。每张图配一个任务相关的提示问题 Q 。设 $\mathcal{M}$ 为当前的生成式 LLM，它把每个 $G-Q$ 对作为输入，返回一条逐步的文本推理路径 R ，即 $R = \mathcal{M}(G, Q)$ 。这条 R 由多个词或句子组成，在自然语言处理里常被称作**思维链（Chain-of-Thought, CoT）**。对某些任务（如最短路）， G 里还会

附带额外信息，比如边的权重 w 。

5 五、数据集 GraphInstruct: 9 类任务与“自增强”造数据

GraphInstruct 是全文的地基。它的两个最大特点是 **Yes (含训练集)** 和 **Yes (含 CoT 推理路径)**——这正是它区别于以往 NLGraph、GraphQA 等数据集的地方（后两者都没有训练集、也没有推理路径）。

5.1 9 类图计算任务（原文 Table 1）

表 1: GraphInstruct 的九类任务概览（数据源自原文 Table 1）。 $|V|$ 、 $|E|$ 为节点数与边数。

任务	要做什么	时间复杂度	节点范
Cycle Detection（找环）	判断图中是否存在环	$O(V + E)$	[2,100]
Connectivity（连通性）	判断两节点间是否有路径	$O(V + E)$	[2,100]
Bipartite Check（二分图）	判断图是否为二分图	$O(V + E)$	[2,100]
Topological Sort（拓扑排序）	求有向无环图的拓扑序	$O(V + E)$	[2,50]
Shortest Path（最短路）	求两节点间最短路径	$O(E + V \log V)$	[2,100]
Maximum Triangle Sum（最大三角形和）	求权重和最大的连通三元组	$O(V ^3)$	[2,25]
Maximum Flow（最大流）	求源点到汇点的最大流量	$O(V ^2 \sqrt{ E })$	[2,50]
Hamilton Path（哈密顿路径）	判断是否存在恰好经过每点一次的路径	NP-Complete	[2,50]
Subgraph Matching（子图匹配）	判断 G 中是否有子图与 G' 同构	NP-Complete	[2,30]

5.2 怎么造出来的：六步“自增强 (self-augment)”流水线

直接让人手工写几万条图论推理路径不现实，作者用了一套自动化流程（原文 Figure 2，含 6 步）：
选任务 → **生成图问题** → **用 GPT-4 初步生成** → **训练“数据增强模型”** → **拒绝采样** → **路径筛选** → **得到 GraphInstruct**。几个关键点：

- **随机造图**：用 **Erdős–Rényi (ER) 随机图模型**（给定节点数 n 和连边概率 p ）配合 NetworkX 库批量生成图，并算出标准答案。初始造了 27,000 道训练题（9 类各 3,000）和 3,600 道测试题。
- **先让 GPT-4 起草**：对每道题用 2-shot 思维链提示让 GPT-4 写推理路径，温度设 0.9、重复 3 次以增加多样性，**只保留答对的**，得到初始集 D_1 （不足 8,000 条）。
- **拒绝采样 (rejection sampling) 补缺**：GPT-4 在某些难任务上答对率很低（如最大流不到 100 条）。于是用 D_1 微调一个**数据增强模型** $\mathcal{M}_1$ （基于 LLaMA 2-13B），让它去**补答 GPT-4 答错的题**，并为同一道题生成**多条不同的推理路径**。

- **多样性筛选**：用编辑距离、TF-IDF、Jaccard 相似度（基于字符串）和 Sentence-BERT 余弦相似度 + K-means 聚类（基于语义）来挑出**既正确又彼此不同的**推理路径，避免大量雷同。

【小白补丁】为什么“一题配多条推理路径”很重要

同一道图论题，往往有多种正确的解法走法。让数据集里一道题对应**多条不同的正确推理链**，能让模型见多识广、学到多样的解题风格，而不是死记一种套路。最终 GraphInstruct 含 17,158 个图-问题对、共 **72,785** 条推理路径——平均一道题约配 4 条路径。

6 六、方法逐节精讲：两阶段训练 (SFT $\rightarrow$ DPO)

有了数据，GraphWiz 的训练分**两个阶段**（见图 2）：第一阶段用**混合任务指令微调 (SFT)** 让模型学会解题；第二阶段用 **DPO 对齐** 让模型学会“偏好正确链、远离错误链”。骨干模型用开源的 LLaMA 2-7B/13B 和 Mistral-7B。

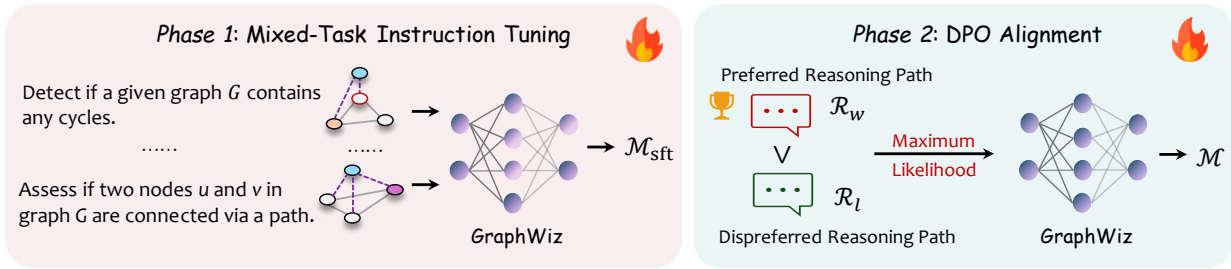


图 2: GraphWiz 的两阶段训练流程（原文 Figure 3，图内英文为原文保留）。左（**Phase 1: 混合任务指令微调**）：把 9 类任务的“图 + 问题”一起喂给模型做 SFT，得到中间模型 $\mathcal{M}_{\text{sft}}$ ；右（**Phase 2: DPO 对齐**）：给模型成对样本——更受偏好的推理路径 $\mathcal{R}_w$ 与不受偏好的 $\mathcal{R}_l$ ，通过“最大似然偏好”训练，让它更喜欢 $\mathcal{R}_w$ ，最终得到 GraphWiz。

6.1 第一阶段：混合任务指令微调 (SFT)

混合任务 (mixed-task) 指的是把 9 类任务**混在一起同时训练**，而不是一类一类分开训。这样做的好处是：一个模型通吃所有任务，且任务之间能**互相促进**（学会找环的能力可能也帮到判连通）。

训练用最普通的**语言模型损失**：把“图 G_i + 问题 Q_i ”当输入，让模型逐词预测正确的推理路径 $R_{i,j}$ ，最小化预测与真实路径之间的差距：

$$\mathcal{L}_{\text{SFT}} = - \sum_{i=1}^N \sum_{j=1}^M \log \mathbb{P}(R_{i,j} \mid G_i, Q_i; \theta). \quad (2)$$

其中 N 是样本数， M 是 G_i - Q_i 对应的推理路径条数， θ 是模型参数。直观理解：让模型给“正确推理路径”里每个词都打出尽量高的概率。这一阶段产出的模型记为 $\mathcal{M}(\theta)_{\text{sft}}$ 。

6.2 第二阶段：DPO 对齐——本文把 DPO 用到图论上

SFT 只让模型“模仿对的”，但难任务上模型仍会犯错。DPO 的思路是：**给它看成对的对错样本，逼它学会区分**。关键难点是：**怎么为每道题找到这一对 R_w （好）和 R_l （坏）**？作者的做法很巧妙：

- R_w （偏好/正确链）：新造 9,000 道题，用上一阶段的 $\mathcal{M}_{\text{sft}}$ 对每题推理 20 次，选其中**最长的那条正确路径**作为 R_w （长链通常推理更完整详细）。
- R_l （不偏好/错误链）：故意挑“**最像正确答案、但其实是错的**”那条（即难负样本 hard-negative）——先用字符串/语义方法找出离 R_w 最近的错误路径，再用**多数投票（Majority Voting）**选出最常出现的错误链。逼模型去分辨这种“似是而非”的陷阱，训练效果更好。

最终收集到 6,166 个 G - Q - R_w - R_l 四元组。DPO 的训练目标函数为：

$$\mathcal{L}_{\text{DPO}}(\mathcal{M}(\theta); \mathcal{M}(\theta)_{\text{sft}}) = -\mathbb{E}_{(x, R_w, R_l) \sim D} \log \sigma \left(\beta \log \frac{\mathcal{M}(\theta)(R_w | x)}{\mathcal{M}(\theta)(R_l | x)} - \beta \log \frac{\mathcal{M}(\theta)_{\text{sft}}(R_w | x)}{\mathcal{M}(\theta)_{\text{sft}}(R_l | x)} \right). \quad (3)$$

其中 x 是“图 G 与问题 Q 的拼接”， σ 是 sigmoid 函数， β 是一个超参数。

【小白补丁】把 DPO 公式 (3) 拆开讲明白

公式里有两个分数比，对照着看就懂：

- $\frac{\mathcal{M}(\theta)(R_w | x)}{\mathcal{M}(\theta)(R_l | x)}$ ：正在训练的模型有多偏爱“好链”相对“坏链”。
- $\frac{\mathcal{M}(\theta)_{\text{sft}}(R_w | x)}{\mathcal{M}(\theta)_{\text{sft}}(R_l | x)}$ ：冻结的 SFT 模型的偏好（当作一个“参照基准”）。

两者相减、乘以 β 、过 sigmoid，再取负号当损失——效果就是：**在不偏离原 SFT 模型太远的前提下，推着新模型相对地更偏好 R_w 、更冷落 R_l** 。 β 控制“偏好对比”的强弱：太大太激进、太小没效果，需要调。这正是 DPO 优雅之处：**不用训练奖励模型，一个损失函数就把“偏好对齐”这件事干了**。

7 七、实验与结果表解读

对比对象分两大类： 上下文学习 (In-Context Learning)，模型参数不变，靠提示词答题——代表是闭源的 GPT-3.5、GPT-4（用零样本和 2-shot 思维链）；**用简单标签训练 (Naive-SFT)**，只给模型“yes/no”直答、不给推理过程；此外还训了 GCN/GIN/GAT 三种图神经网络 (GNN) 作对比。

7.1 主结果：GraphWiz 全面超过 GPT-4（原文 Table 4 关键行重排）

表 2: 主结果对比（准确率%，数值源自原文 Table 4；越高越好，每列加粗为该组最优）。“Average” 为九任务平均。

类别	方法	找环	连通	拓扑排序	哈密顿	Average
闭源模型	GPT-4 (zero-shot)	38.75	17.00	5.00	59.25	27.67
	GPT-3.5 (2-shot)	51.25	43.75	4.50	54.25	32.97
	GPT-4 (2-shot)	52.50	62.75	25.25	75.75	43.81
Mistral-7B	Naive SFT	73.75	83.50	1.00	31.75	46.56
	GraphWiz	92.00	89.50	19.00	26.50	53.75
	GraphWiz-DPO	85.50	79.50	85.25	62.75	58.22
LLaMA 2-7B	Naive SFT	73.75	83.50	4.00	69.00	44.81
	GraphWiz	91.50	87.00	18.00	52.25	55.08
	GraphWiz-DPO	89.00	82.50	46.75	81.50	65.00
LLaMA 2-13B	Naive SFT	73.75	83.75	0.50	59.75	44.69
	GraphWiz	94.75	87.00	28.00	59.00	57.39
	GraphWiz-DPO	87.50	88.50	72.75	46.50	63.89

怎么读这张表：

- **核心数字**：最强的 **GraphWiz-DPO (LLaMA 2-7B)** 平均 **65.00%**，远超 **GPT-4 (2-shot)** 的 **43.81%**（高出约 21 个百分点）。这就是摘要里 “65% vs 43.8%” 的来历。一个 7B 的开源小模型，平均表现把 GPT-4 甩在身后。
- **DPO 确实有用**：在三种骨干上，GraphWiz-DPO 的平均分都比纯 GraphWiz (SFT) 更高（如 LLaMA 2-7B 从 55.08 升到 65.00）。在**拓扑排序**这类难任务上提升最明显（Mistral 从 19.00 飙到 85.25）。
- **但 DPO 不是“万灵药”**：它在个别任务上反而掉分（如 Mistral 找环从 92.00 降到 85.50），说明 DPO 总体提升推理、但对某些任务仍需进一步调参。
- **“写过程”比“只给答案”强**：GraphWiz 远超 Naive-SFT，证明**让模型显式推理**本身就能提升正确率，而不只是为了可解释性。

7.2 其他重要发现

- **数据量并非越多越好 (RQ2, 原文 Table 5)**：随着“图: 推理路径”比例 G:R 从 1:1 加到 1:5，平均分总体上升（LLaMA 2-7B: 46.56 → 55.08）；但**个别任务会过拟合**——如 Mistral 的最大三

角形和从 1:1 的 47.00% 下降到 1:5 的 38.75%。提示：堆数据存在阈值，需配合验证集监控。

- **很能“举一反三”（RQ3）**：作者训了一个只学两类 NP 难任务（哈密顿、子图匹配）的变体 GraphWiz-High，发现它在从没训练过的简单/中等任务上也能零样本迁移、明显超过原始骨干，体现跨任务泛化能力。
- **图越大越难（RQ4，原文 Figure 6）**：GraphWiz 和 GPT-4 都随图变大而掉分，但 GraphWiz 在大多数规模上稳定领先；不过当节点数**超过 60**后两者都几乎失效——因为它们主要靠“枚举”推理，规模一大枚举量指数爆炸。
- **案例对比（原文 Figure 8）**：在一张 89 节点的找环题上，GPT-4 直接给出一个很短的错误答案“没有环”，而 GraphWiz 正确找出一条具体的环路并写出完整路径，展现了对大规模复杂图的空间推理与记忆能力。

8 八、小白知识点卡片（速查）

- **图计算问题**：靠图算法求解的题，如连通性、最短路、找环、拓扑排序、最大流、哈密顿路径等。
- **显式推理路径 / 思维链（CoT）**：模型把“怎么想出来的”一步步写成文字，而不仅给最终答案。
- **指令微调 / SFT**：用“指令 + 标准答案”训练模型听话做事；SFT 是其最常见的监督训练形式。
- **DPO（直接偏好优化）**：给成对的“好/坏”回答，用一个损失函数直接教模型偏好好的、远离坏的；RLHF 的轻量替代。
- **RLHF / PPO**：基于人类反馈的强化学习；PPO 是其常用算法，但要 4 个子模型、成本高。
- **NP 完全（NP-Complete）**：一类目前无已知高效算法、规模一大就极难的问题。
- **ER 随机图**：用“节点数 n + 连边概率 p ”随机生成图的经典模型。
- **拒绝采样（rejection sampling）**：反复采样、只保留满足条件（这里是“答对”）的结果来扩充数据。
- **难负样本（hard-negative）**：长得最像正确答案、却是错的样本；用它训练能逼模型学得更细。
- **过拟合（overfitting）**：模型死记训练数据的细节，反而在新题上变差。

9 九、一句话总结与谱系定位

一句话：GraphWiz 先用“GPT-4 起草 + 小模型自增强”造出带显式推理链的图论指令数据集 GraphInstruct，再**两阶段训练**——先 SFT 学会解 9 类图论题、写出推理过程，后用 DPO 让模型**偏好正确链、远离错误链**，最终用一个 7B 开源模型把平均准确率做到 65%，反超 GPT-4 的 43.8%。

在“图 × 后训练”谱系里的位置：它是“指令微调 + 偏好对齐 (SFT+DPO)”路线在图计算推理方向的代表作，核心卖点是**显式推理 + 开源 + 通吃多任务**。与本系列其他工作对照看更清楚：G-Retriever 走“检索 + 软提示 (GraphRAG)”、LLaGA 走“投影器对齐”、G1 走“强化学习”，而 GraphWiz（与同期的 InstructGraph）走的是“**造高质量推理数据 + SFT + DPO**”这条路——你可以对照着体会它们各自把“图的推理能力”塞进 *LLM* 的方式有何不同。

动手复现：官方代码与数据见 <https://github.com/nuochenpku/Graph-Reasoning-LLM>，项目主页 <https://graph-wiz.github.io>。其中 GraphInstruct 数据集本身就是入门“LLM 图推理”很好的练手材料。